Ringle AI Tutor

Coding Agent Interaction History

AI 기반 개발 툴 활용 문서 | Claude (Anthropic)

도구	Claude (Anthropic) — claude.ai 웹 인터페이스
프로젝트	Ringle AI Tutor Fullstack (Rails 7.1 + React 18 + Docker)
활용 방식	대화형 프롬프트 입력 → 코드 생성 → 검토·수정·검증 반복
주요 영역	Rails API, React/TypeScript, OpenAI 연동, Docker, RSpec 테스트

1. 개요 및 사용 도구

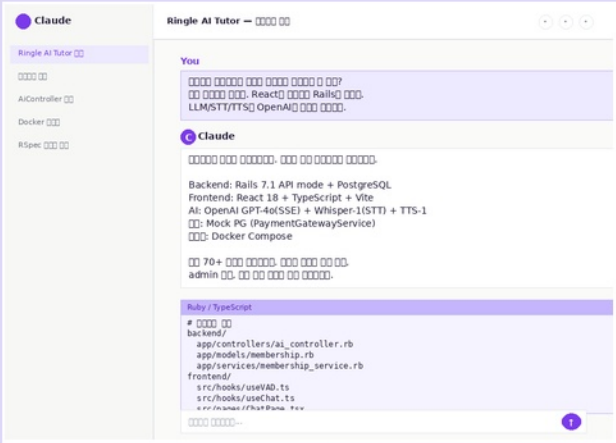
본 프로젝트는 링글 AI 튜터 풀스택 구현과제입니다. 개발 전 과정에서 Claude (Anthropic)를 AI 기반 개발 도구로 활용했으며, claude.ai 웹 인터페이스를 통해 대화형으로 프롬프트를 입력하고 생성된 코드를 검토·수정하며 구현을 완성했습니다.

2. AI 도구 활용 워크플로우

단계	활동
① 요구사항 분석	과제 PDF를 읽고 구현 항목을 정리한 뒤 Claude에 전달
② 아키텍처 설계	기술 스택(Rails + React + PostgreSQL)과 전체 구조 설계 요청
③ 코드 생성	기능별로 세분화된 프롬프트로 코드 생성
④ 코드 검토	생성된 코드를 직접 읽으며 비즈니스 로직·보안 검토
⑤ 실행·테스트	Docker로 실행 후 오류 로그를 Claude에 붙여넣어 디버깅
⑥ 수정·개선	발견된 문제(버그, 설계 오류)를 Claude와 함께 수정

3. 주요 인터랙션 사례 — 인터랙션 #1

전체 아키텍처 설계 및 프로젝트 구조 수립



▲ claude.ai 화면: 전체 아키텍처 설계 요청 및 프로젝트 구조 생성

☒ PROMPT (입력)

과제 PDF를 공유하며: "사전과제 요구사항을 꼼꼼히 살펴보고 만들어줄 수 있어?
진짜 꼼꼼하게 부탁해. React는 익숙하고 Rails는 낯설다.
LLM/STT/TTS는 OpenAI를 사용할 예정이다."

☒ OUTPUT (출력 요약)

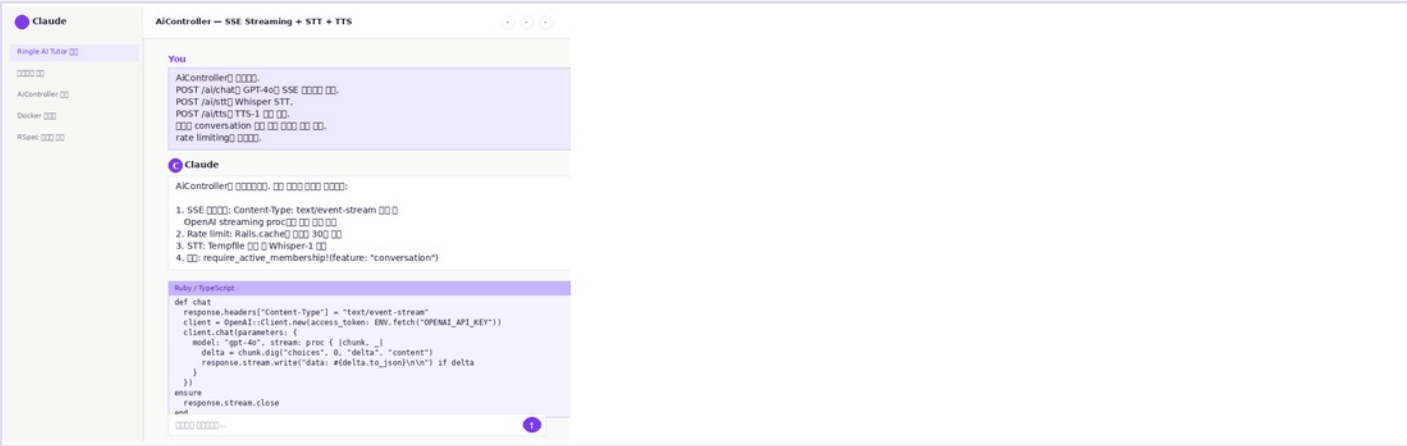
Rails 7.1 API mode + React 18 + Docker Compose 구성.
4개 마이그레이션(users, membership_plans, memberships, payments),
MembershipService, PaymentGatewayService(Mock PG) 포함 70+ 파일 생성.

☒ 검토·수정 내용

(1) ActionController::Live(SSE) 미들웨어 설정 누락 발견 → application.rb에 추가 요청.
(2) Membership lazy expiry 전략 검토 후 Sidekiq 권장 주석 추가.
(3) UUID primary key 설정이 migration에 올바르게 적용되었는지 확인.

3. 주요 인터랙션 사례 — 인터랙션 #2

AiController — SSE Streaming + STT + TTS 구현



▲ claude.ai 화면: AiController SSE 스트리밍 구현 코드 생성

☒ PROMPT (입력)

"AiController를 만들어줘. POST /ai/chat는 GPT-4o SSE 스트리밍,
POST /ai/stt는 Whisper STT, POST /ai/tts는 TTS-1 음성 합성.
멤버십 conversation 기능 있는 유저만 사용 가능, rate limiting도 적용해줘."

☒ OUTPUT (출력 요약)

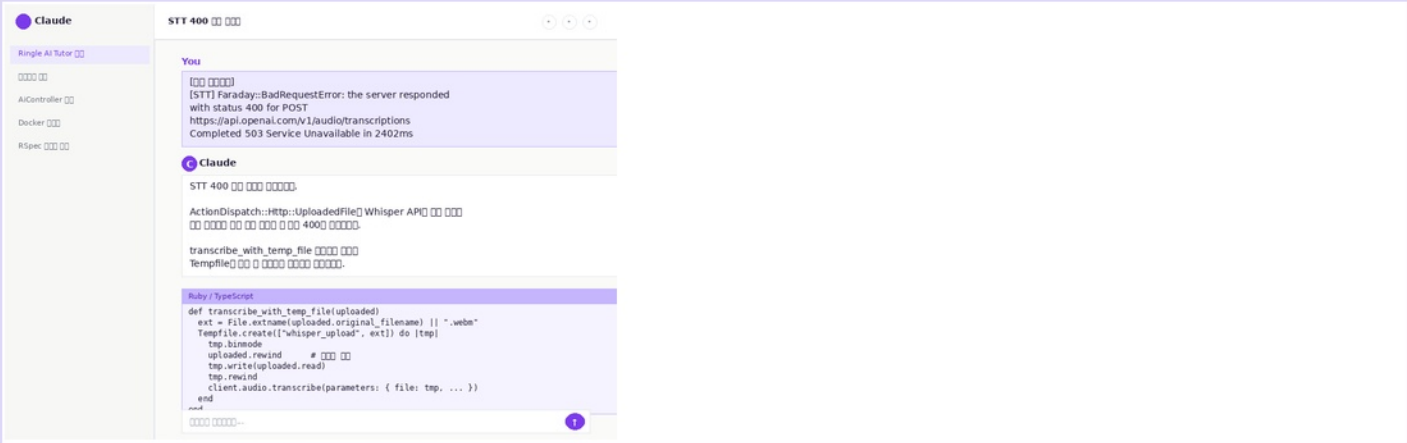
SSE 헤더(Content-Type: text/event-stream) 설정 후 OpenAI streaming proc으로 토큰 단위 전송.
Rails.cache로 시간당 30회 rate limit.
require_active_membership!(feature: "conversation")으로 권한 제어.

☒ 검토·수정 내용

(1) STT에서 UploadedFile 직접 전달 → Faraday::BadRequestError 400 발생.
Tempfile 변환 메서드(transcribe_with_temp_file) 추가 요청.
(2) SSE 에러 메시지 따옴표 중첩 문법 오류 → 상수(SSE_ERROR_GENERIC)로 분리 해결.

3. 주요 인터랙션 사례 — 인터랙션 #3

STT 400 에러 디버깅 — 오류 로그 붙여넣기



▲ claude.ai 화면: STT 400 에러 로그 붙여넣기 및 원인 진단

☒ PROMPT (입력)

[Docker 로그 붙여넣기]
"[STT] Faraday::BadRequestError: the server responded with status 400
for POST https://api.openai.com/v1/audio/transcriptions
Completed 503 Service Unavailable in 2402ms"

☒ OUTPUT (출력 요약)

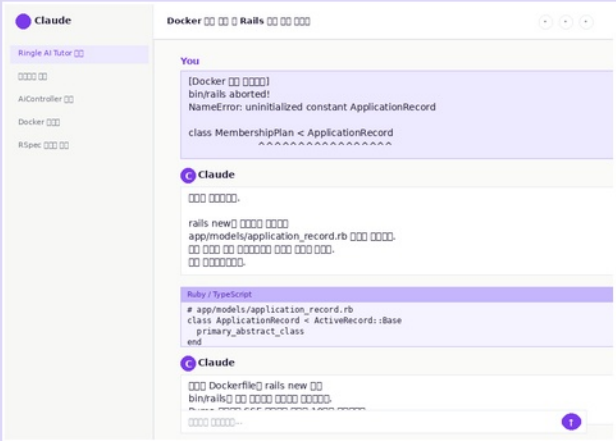
원인: ActionDispatch::Http::UploadedFile을 Whisper API에 직접 전달 시
파일 포인터가 이미 읽힌 상태. Tempfile.create로 명시적 복사 후 전달하는
transcribe_with_temp_file 메서드 생성.

☒ 검토·수정 내용

수정 코드를 Docker 컨테이너 안에서 직접 실행하여 STT 200 OK 응답 확인.
파일 크기 최솟값(100 bytes) 검증 로직도 추가 요청.

3. 주요 인터랙션 사례 — 인터랙션 #4

Docker 환경 구성 및 Rails 실행 오류 디버깅



▲ claude.ai 화면: Docker 오류 로그 붙여넣기 및 단계별 원인 진단

☒ PROMPT (입력)

"설치에 어려움이 있어, Docker에서 돌아갈 수 있도록 수정해줘."
(이후) [로그] "NameError: uninitialized constant ApplicationRecord"
(이후) "rails new가 실패해. exit code 1"

☒ OUTPUT (출력 요약)

1단계: app/models/application_record.rb 누락 → 추가.
2단계: rails new 불안정 → bin/rails 직접 생성 방식으로 Dockerfile 재설계.
3단계: docker-entrypoint.sh에 PostgreSQL 연결 대기 + 자동 마이그레이션 추가.

☒ 검토·수정 내용

Puma 스레드 수를 5→10으로 증가 (SSE 스트리밍 스레드 점유 특성 고려).
PostgreSQL 준비 전 Rails가 실행되는 race condition 직접 발견
→ until 루프 추가 요청.

3. 주요 인터랙션 사례 — 인터랙션 #5

RSpec 테스트 31개 실패 원인 분석 및 수정

▲ claude.ai 화면: RSpec 31개 실패 로그 분석 및 4가지 근본 원인 도출

☒ PROMPT (입력)

RSpec 실행 결과 전체를 붙여넣으며:
"테스트에서 문제가 발생했어. 어떻게 해야 하는지 알려줘."
(31개 실패: factory 검증 실패, 모든 요청 403 Forbidden, Payment.last nil)

☒ OUTPUT (출력 요약)

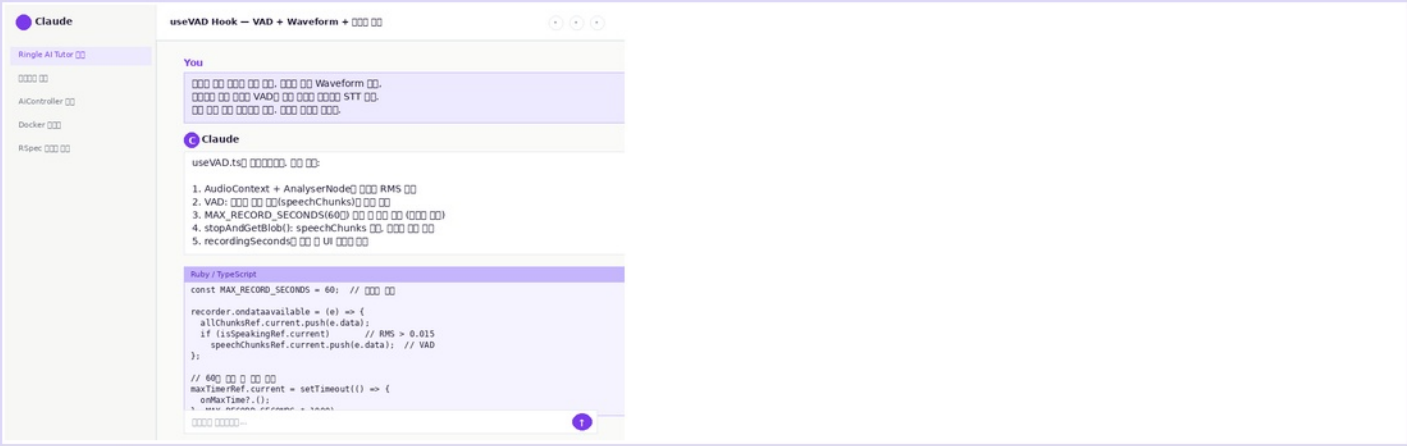
원인 1: expired factory의 starts_at 미설정 → validation 실패 (1개)
원인 2: uniqueness: { case_insensitive: true } 잘못된 옵션명 (1개)
원인 3: require_user!에 performed? 가드 누락 → 전체 403 반환 (26개)
원인 4: 트랜잭션 내 payment.update!(failed)가 raise와 함께 rollback (2개)

☒ 검토·수정 내용

(1) performed? 가드는 ActionController::Live 병용 시 동작 차이 있어 직접 확인.
(2) ai_spec.rb의 receive_message_chain 오류 → instance_double 방식으로 개선.
(3) MembershipService 트랜잭션 분리가 설계상 올바름을 검토 후 적용.

3. 주요 인터랙션 사례 — 인터랙션 #6

VAD + Waveform + 답변완료 버튼 (useVAD Hook)



▲ claude.ai 화면: useVAD Hook VAD + 오남용 방지 구현

☒ PROMPT (입력)

"마이크 버튼 누르면 녹음 시작, 말하는 동안 Waveform 표시,
답변완료 버튼 누르면 VAD로 공백 제거한 오디오를 STT 전송.
자동 종료 없이 버튼 제출 방식. 오남용 방지도 필요해."

☒ OUTPUT (출력 요약)

AudioContext + AnalyserNode RMS 계산으로 실시간 Waveform.
말소리 청크(speechChunks)만 별도 저장으로 VAD 구현.
MAX_RECORD_SECONDS(60초) 초과 시 자동 제출 콜백으로 오남용 방지.

☒ 검토·수정 내용

(1) 초기 자동 VAD 종료 방식 → 요구사항 재확인 후 답변완료 버튼 방식으로 변경.
(2) useChat의 isLoading 클로저 버그(stale 값 캡처) 직접 발견
→ isLoadingRef 추가하여 직접 수정.

4. 코드 검토·수정·검증 프로세스

검토 항목	확인 방법	발견 문제 및 수정
비즈니스 로직	요구사항 PDF 대조	답변완료 버튼 방식 누락 → VAD Hook 재설계
보안·권한	코드 직접 리뷰	performed? 가드 누락 → 26개 테스트 실패
API 동작	curl + 브라우저	STT 400 → Tempfile 변환으로 해결
테스트 실행	bundle exec rspec	31개 실패 → 4가지 근본 원인 분류 후 수정
트랜잭션 무결성	서비스 코드 분석	Payment rollback → 트랜잭션 밖으로 이동
Docker 실행	compose logs 확인	application_record.rb 누락, Puma 스레드 부족

5. AI 도구 활용 성과 및 결론

개발 속도 향상	Rails 미숙련 상태에서도 Claude의 도움으로 Rails 7.1 API 앱을 단시간에 구현. 모델·컨트롤러·서비스·마이그레이션을 체계적으로 생성.
디버깅 효율화	오류 로그를 Claude에 붙여넣으면 원인과 해결책을 빠르게 도출. RSpec 31개 실패를 4가지 근본 원인으로 분류한 사례가 대표적.
주의점	생성된 코드를 그대로 사용하면 오류 발생. 항상 직접 실행·테스트로 검증하는 과정이 필수.
AI 도구의 역할	코드 생성 + 디버깅 보조 도구로 활용. 설계 결정은 개발자가 직접 판단 후 Claude에 반영 요청.

Claude(Anthropic)를 활용해 Rails 백엔드·React 프론트엔드·Docker·RSpec 테스트를 포함한 풀스택 앱을 구현했습니다. AI 도구는 코드 생성과 디버깅에서 강력한 생산성을 제공했지만, 생성된 코드의 정확성은 반드시 개발자가 직접 검토·실행·테스트로 검증해야 함을 확인했습니다.